MSDS 535 Week 3

Chapter 3: Data warehousing and online analytical processing

Han, Jiawe, et. al. (2022). Data Mining Concepts and Techniques. Elsevier.

Overview and Objectives

- Discuss Data warehouse.
- Analyze Data warehouse modeling: schema and measures.
- Understand OLAP operations.
- Analyze Data cube computation.
- Understand Data cube computation methods.
- Summarize Data warehouse.

Data Warehousing and Business Intelligence

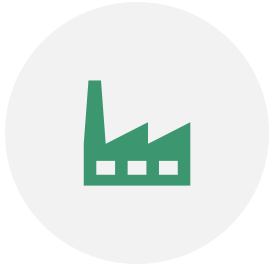
In this section, we explore the concept of data warehousing and its importance.

We'll also discuss data warehouse architecture and the role of data lakes.

What is a Data Warehouse?

- A data warehouse is a dedicated repository for analysis, separate from operational databases.
- It provides a historical perspective on data, supporting decision-making processes.
- Data warehouses are crucial for strategic decision-making and customer retention.

Key Features of Data Warehouses



SUBJECT-ORIENTED: DATA WAREHOUSES FOCUS ON MAJOR SUBJECTS SUCH AS CUSTOMERS, PRODUCTS, AND SALES.

INTEGRATED: THEY INTEGRATE DATA FROM VARIOUS SOURCES AND ENSURE CONSISTENCY.

TIME-VARIANT: DATA WAREHOUSES STORE HISTORICAL DATA, OFTEN SPANNING SEVERAL YEARS.

NONVOLATILE: DATA WAREHOUSES ARE SEPARATE FROM OPERATIONAL SYSTEMS AND TYPICALLY DO NOT DELETE DATA.

Purpose of Data Warehousing

- Data warehouses help organizations make strategic decisions and analyze data for insights.
- They support various analyses, including customer behavior and product sales.
- Analytic results are presented through periodic or ad hoc reports for decision-makers.

Operational vs. Data Warehouses

- Operational databases handle day-to-day operations, while data warehouses serve analysts and decision-makers.
- Operational databases manage current data, while data warehouses store historical, aggregated data.
- Database design and access patterns differ significantly between the two systems.

Benefits of Separating Databases

- Separating operational and data warehouse databases ensures high performance for both.
- Operational databases are optimized for transactions, while data warehouses handle complex OLAP queries.
- Concurrency control and recovery mechanisms can impact OLAP query performance in operational databases.

Data Warehouses

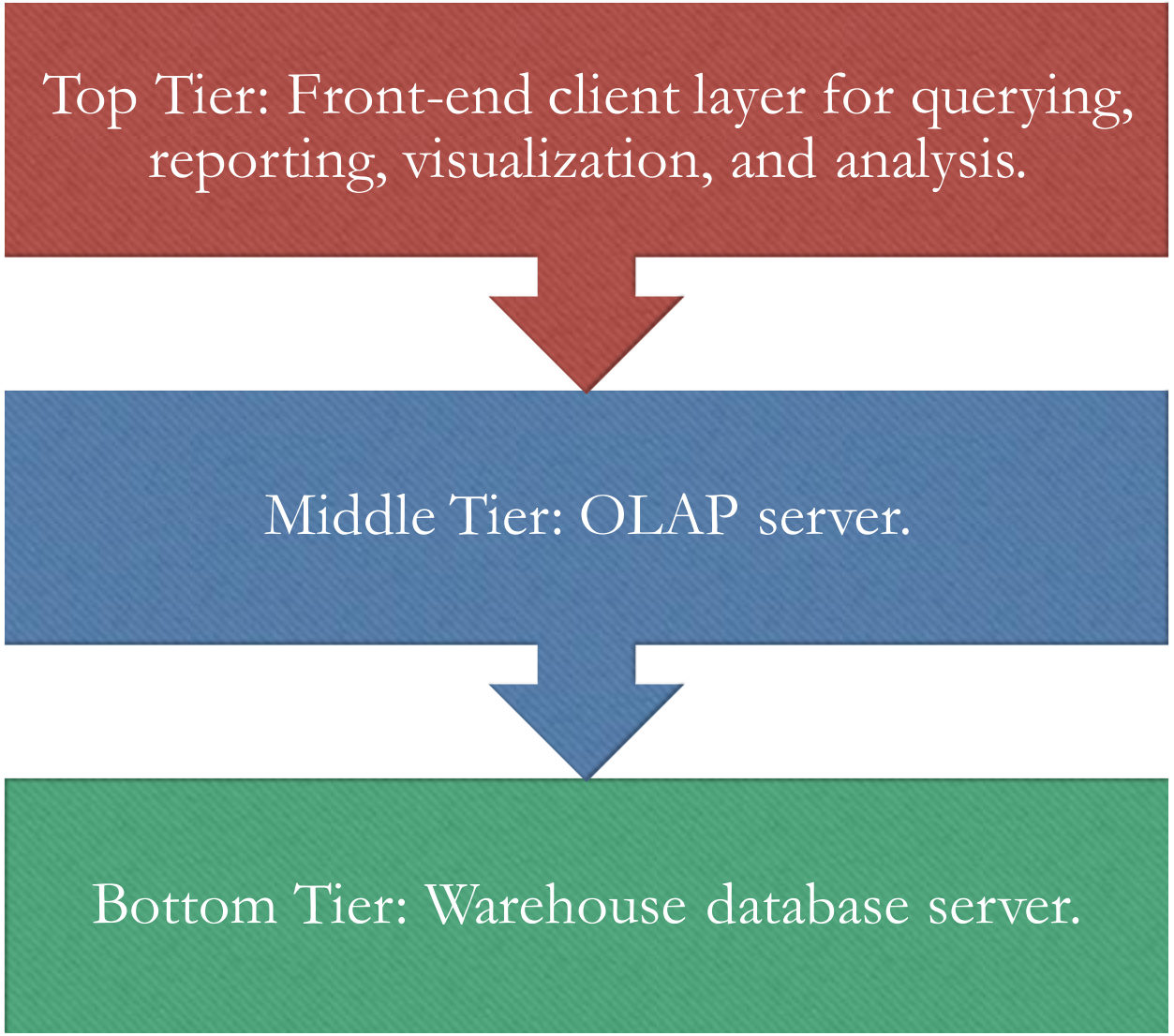
Data warehouses are essential for informed decision-making and business intelligence.

Separating operational and data warehouse databases is crucial for maintaining performance and data quality.

Data Warehouse Architecture

- Explore the architecture of data warehouses, including the three-tier model and two key data warehouse models.
- Three-Tier Architecture
- Metadata in Data Warehouses
- ETL for Data Warehouses
- Enterprise Data Warehouse vs. Data Mart
- AI and Data Warehouses
- Integration of AI and Data Warehouses

Three-Tier Architecture



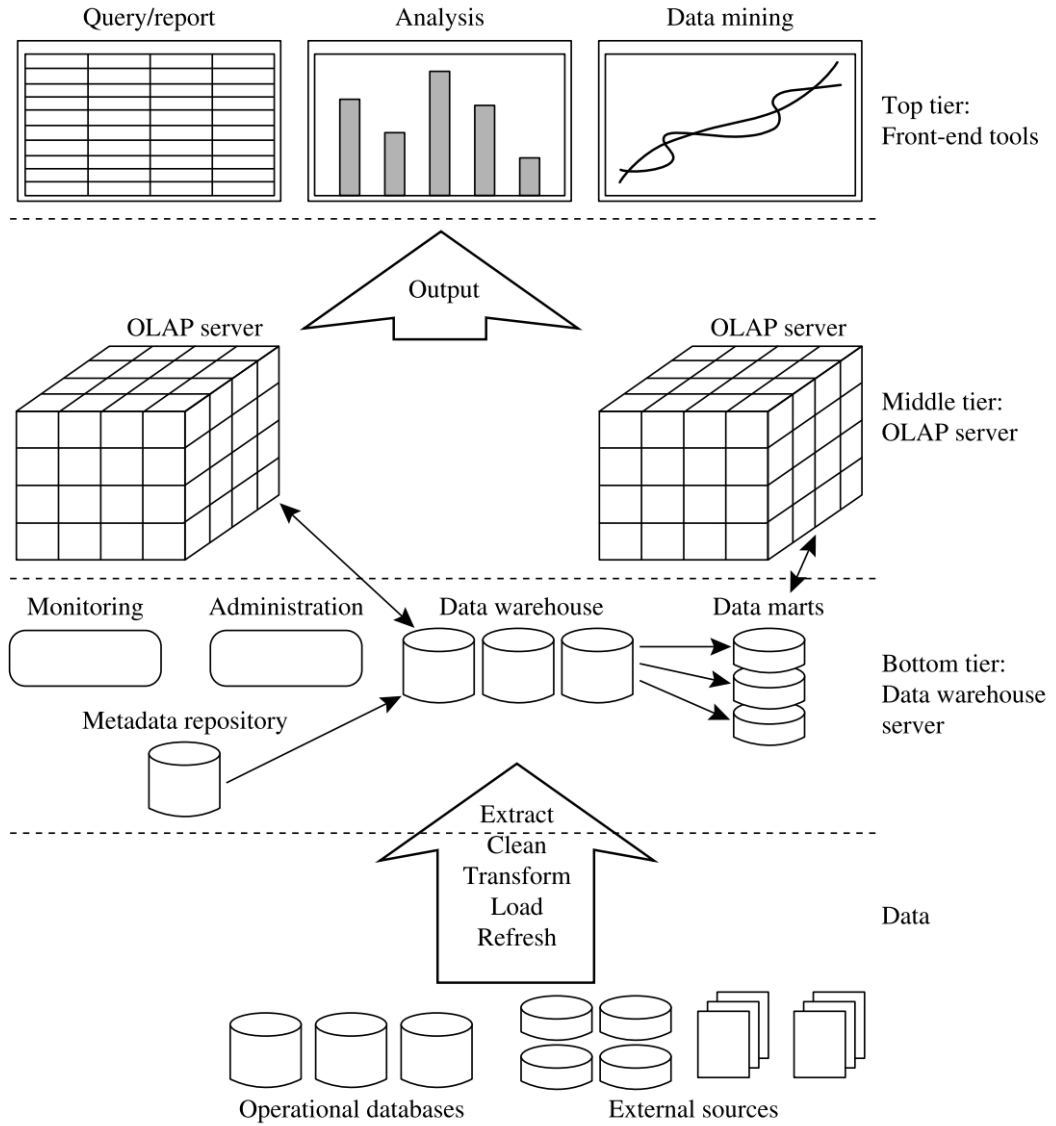


FIGURE 3.1

A three-tier data warehousing architecture.

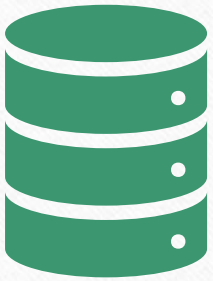
Metadata in Data Warehouses

Metadata are data about data, defining warehouse objects.

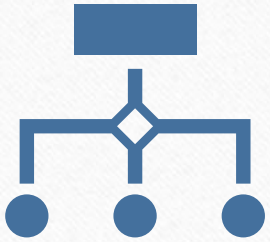
They include data names, definitions, timestamping, source, and more.

Metadata play a crucial role in data mapping, data quality, and data transformation.

ETL for Data Warehouses



ETL (Extraction, Transformation, Loading) modules populate and refresh data warehouses.

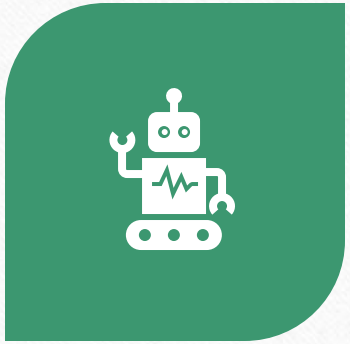


Major ETL tasks include data extraction, data transformation, and data loading.

Enterprise Data Warehouse vs. Data Mart

- Enterprise Warehouse: Cross-functional, corporate-wide data integration.
- Data Mart: Subset for specific user groups, often summarized, quicker implementation.
- Dependent data marts source from enterprise data warehouses.

AI and Data Warehouses



DATA WAREHOUSES SUPPORT AI AND MACHINE LEARNING FUNCTIONALITIES.

AI AND MACHINE LEARNING TECHNIQUES ARE USED IN DATA WAREHOUSES FOR DATA CLEANING, OPTIMIZATION, AND KNOWLEDGE DISCOVERY.

AI MODELS CAN PREDICT LABELS AND ENHANCE DATA WAREHOUSES' ANALYTICAL CAPABILITIES.

Integration of AI and Data Warehouses

- Machine learning techniques enhance data warehouses and optimize performance.
- AI tools assist knowledge workers in exploring data and making informed decisions.
- Conclusion
  - Data warehouse architecture includes three tiers and two primary models: enterprise warehouse and data mart.
  - AI and machine learning enhance data warehouses and support knowledge discovery.

Introduction to Data Lakes and Data Warehouses

Data lakes are centralized repositories for diverse data types.

They complement or serve as alternatives to data warehouses.

Challenges in managing complex data sources in large organizations include data variety, format, and quality.

Data lakes aim to address these challenges and provide flexibility for data-driven analysis.

Data lakes and data warehouses are crucial components of modern data management strategies.

Data Lakes: Concept and Characteristics

- Data lakes serve as single repositories for data in natural formats.
- They store various data types, including relational, semi-structured, unstructured, and binary data.
- Data lakes are often hosted in the cloud or distributed data repositories.
- Both raw and transformed data can be stored in data lakes.
- Data lakes support a wide range of analytical tasks, including reporting, visualization, analytics, and data mining.

Differences Between Data Warehouses and Data Lakes

- Data warehouses are designed for structured, top-down data analysis.
- Data lakes retain all data in their natural form, allowing bottom-up, flexible analysis.
- Data lakes encompass both historical and current data for broader usage.
- Data warehouses are typically used by data analysts and executives.
- Data lakes are accessible by a broader range of users, including operational users and data scientists.

Advantages of Data Lakes

- Data lakes provide flexibility for one-time or exploratory data analysis.
- Self-service business intelligence empowers data scientists to work directly with data.
- Data lakes can cover a broader spectrum of users and business needs.
- They are always available for exploring novel data usages.
- Data lakes efficiently support diverse data types and formats.

Data Lake Architecture

- Data lakes ingest data from various sources through connectors.
- Data flows through layers, including raw data, standardized data, cleansed data, and application data layers.
- Business logic is implemented at the application data layer.
- Users can discover data sets using an enterprise search engine.
- Data services and APIs facilitate application development and reporting.

Data Lake Components Beyond Storage

Security is crucial, with defined access, authentication, authorization, and data protection.

Governance includes monitoring, logging, and lineage.

Data quality, auditing, archives, and stewardship are essential aspects.

Availability, usability, security, and integrity are monitored and managed.

Data scientists and analysts may create new data in the optional sandbox data layer.

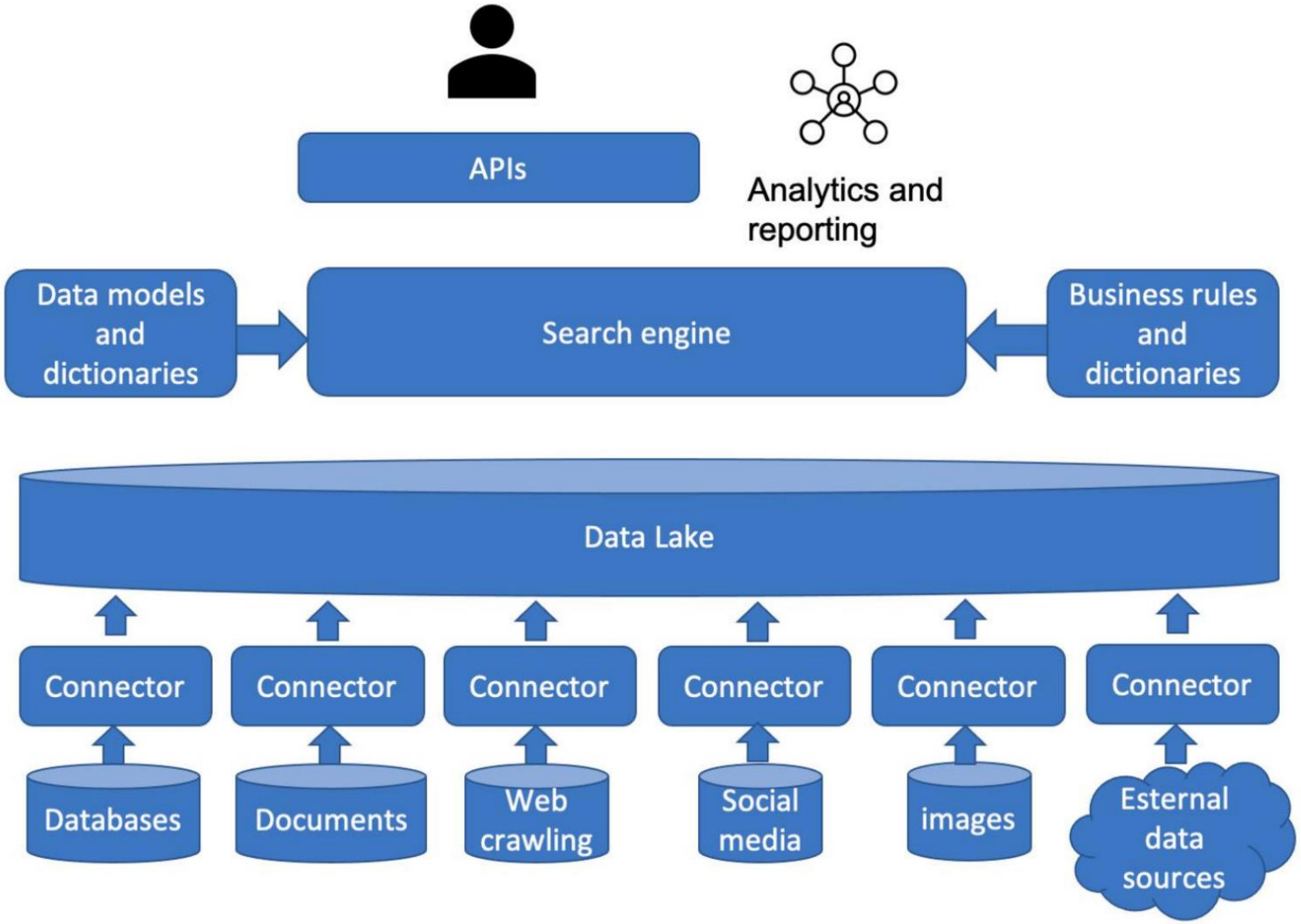


FIGURE 3.3

The conceptual architecture of data lakes.

Data Warehouse Modeling: Schema Types

- Multidimensional data models organize data around central themes.
- Star schemas feature a central fact table and dimension tables.
- Snowflake schemas normalize dimension tables for reduced redundancy.
- Fact constellation schemas involve multiple fact tables sharing dimension tables.
- Schema design influences data retrieval and analysis.

Star Schema

- Star schema features a centralized fact table.
- Fact table contains keys to dimension tables and measures.
- Dimension tables represent different perspectives, e.g., time, item, location.
- This schema simplifies data retrieval and supports efficient queries.
- Fact table keys link to dimension tables for richer analysis.

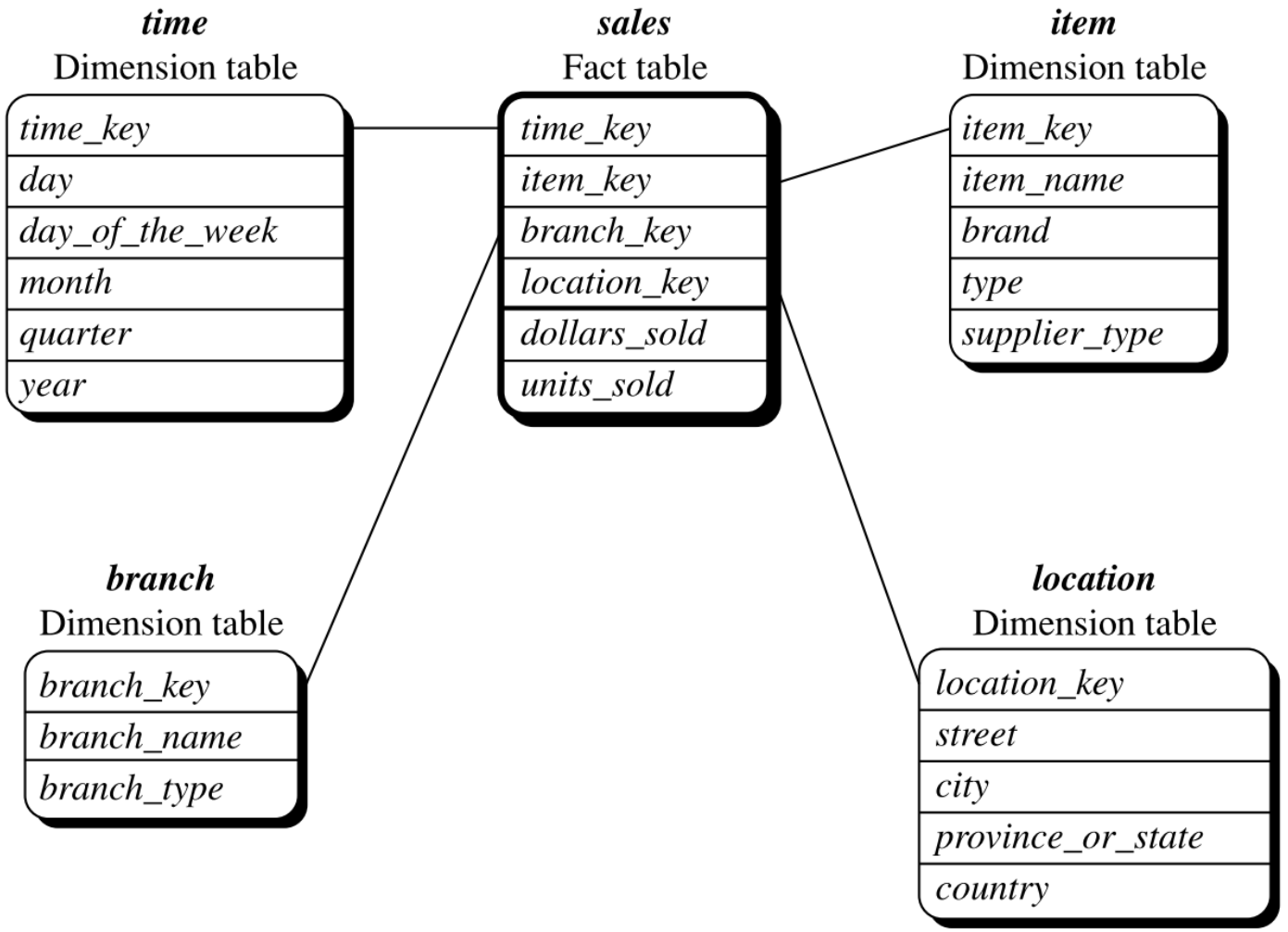


FIGURE 3.7

Star schema of *sales* data warehouse.

Snowflake Schema

- Snowflake schema normalizes some dimension tables.
- Normalization reduces redundancy in dimension tables.
- Schema design improves data integrity but may complicate queries.
- It's a variant of the star schema with some tables broken into sub-tables.
- Useful when data integrity is a top priority.

Fact Constellation

Fact constellation involves multiple fact tables.

Fact tables share dimension tables.

Enables complex analyses by connecting multiple subject areas.

Provides flexibility for a wide range of business questions.

Requires careful schema design to ensure efficient query performance.

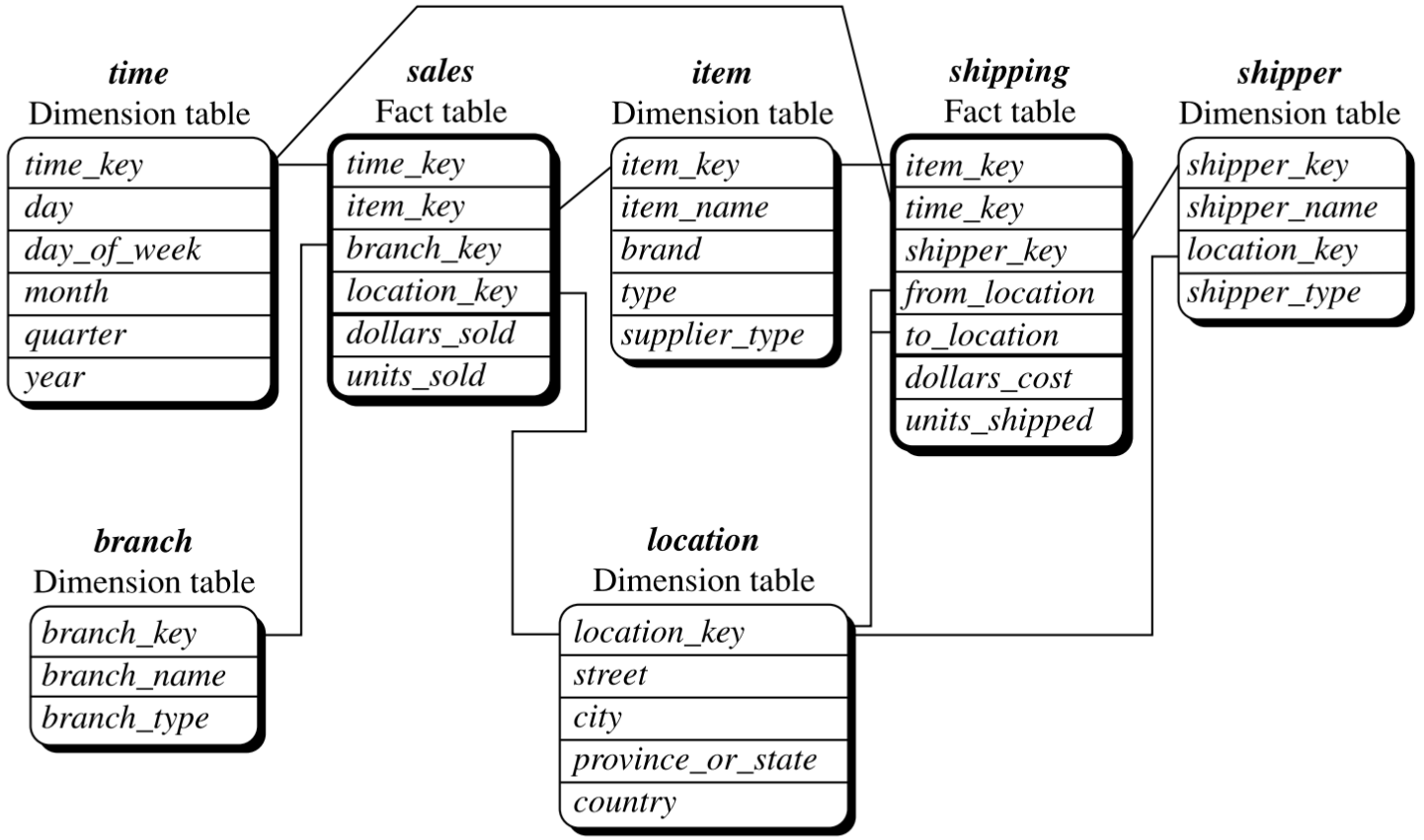


FIGURE 3.9

Fact constellation schema of a sales and shipping data warehouse.

Concept Hierarchies

Concept hierarchies define mappings from low-level to high-level concepts.

Hierarchies are based on dimensions and attributes.

They allow data to be handled at varying levels of abstraction.

Users can customize or define hierarchies based on their needs.

Hierarchies can be predefined or generated based on data distribution.

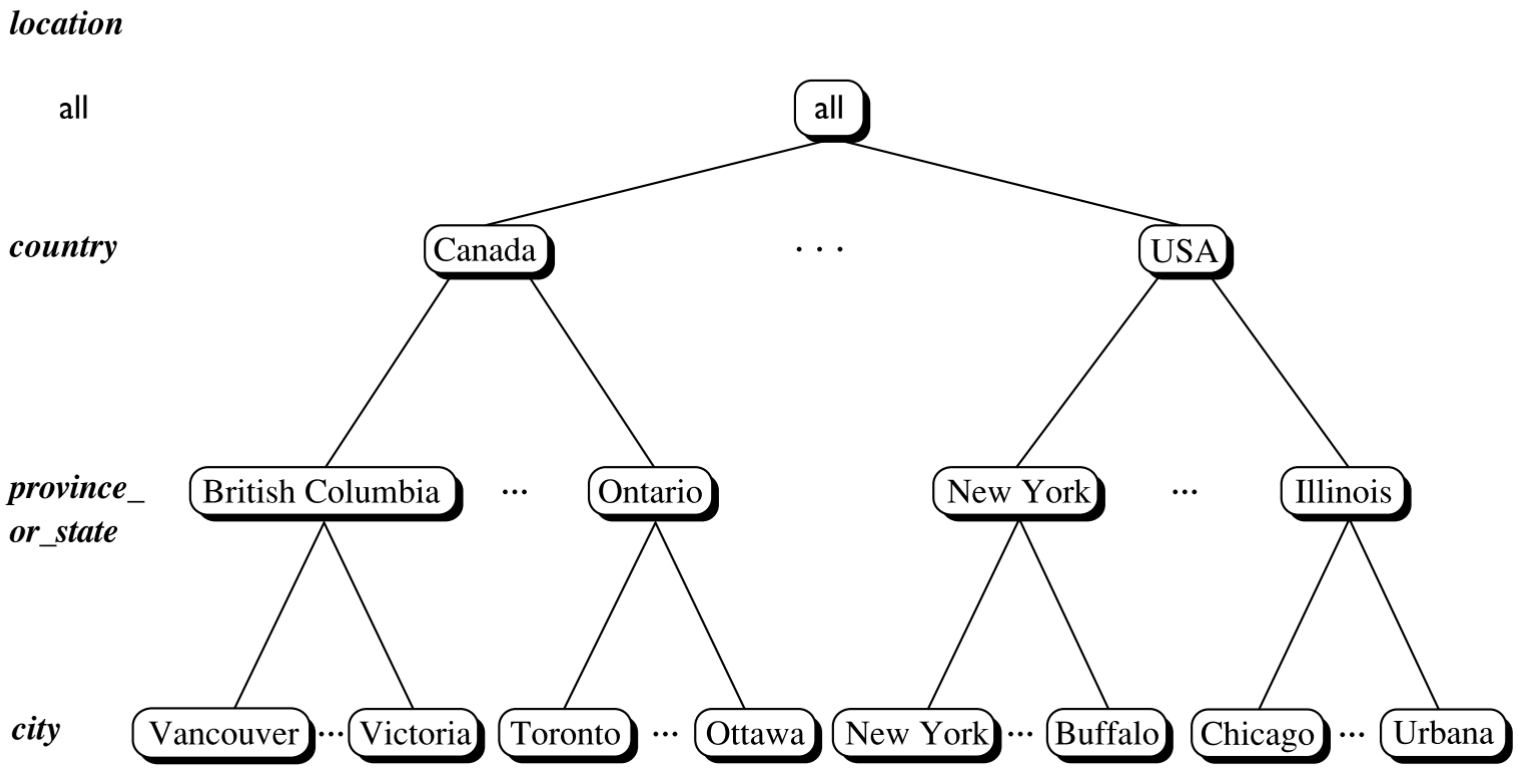


FIGURE 3.10

A concept hierarchy for *location*. Due to space limitations, not all of the hierarchy nodes are shown, indicated by ellipses between nodes.

Measures Categories

- Measures are categorized into distributive, algebraic, and holistic types.
- Distributive measures can be efficiently computed through partitioning.
- Examples include sum(), count(), min(), and max().
- Algebraic measures are computed using algebraic functions with distributive measures.
- Measures play a crucial role in data cube analysis.

Distributive Measures

- Distributive measures can be computed by aggregating partitions.
- Examples include sum() for total sales and count() for counting records.
- Efficient for large data sets as computations can be parallelized.
- They are fundamental for data cube analysis.
- Min() and max() are also distributive measures.

Algebraic Measures

- Algebraic measures are computed using algebraic functions with distributive measures.
- Examples include avg() (average), min_N(), max_N(), and standard_deviation().
- They are efficient and scalable for data cube analysis.
- Computed using combinations of distributive functions.
- Offer valuable insights into data trends and patterns.

Holistic Measures

Holistic measures are complex and don't have constant bounds for subaggregates.

Examples include median(), mode(), and rank().

Efficient computation is challenging.

Approximation techniques may be necessary for large datasets.

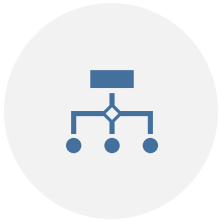
Holistic measures provide specialized insights when needed.

Introduction to OLAP Operations

- Data warehouses are specialized systems designed to support complex analytical queries and reporting.
- OLAP, which stands for Online Analytical Processing, is a crucial aspect of data warehousing.
- In this section, we'll delve into various OLAP operations that facilitate interactive data analysis.
- These operations allow users to explore data from different angles and dimensions, making it user-friendly.
- To illustrate these concepts, we'll use Example 3.4, which showcases key OLAP operations in a practical scenario.

OLAP Operations Overview

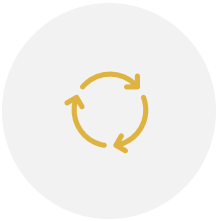
OLAP, IN ITS ESSENCE, ORGANIZES DATA INTO MULTIPLE DIMENSIONS, CREATING A MULTIDIMENSIONAL MODEL.



WITHIN EACH DIMENSION, YOU HAVE HIERARCHIES, WHICH ARE LIKE LEVELS OF ABSTRACTION, HELPING YOU VIEW DATA AT VARYING LEVELS OF GRANULARITY.

THE PRIMARY GOAL OF OLAP OPERATIONS IS TO PROVIDE USERS WITH THE FLEXIBILITY TO INTERACTIVELY ANALYZE DATA.

USERS CAN LOOK AT DATA FROM DIFFERENT PERSPECTIVES AND ADJUST THEIR QUERIES ON-THE-FLY.



THESE OPERATIONS ARE FUNDAMENTAL FOR DYNAMIC DATA EXPLORATION.

Key OLAP Operations

- Roll-up: This operation involves aggregating data by moving up the concept hierarchy of a dimension. For instance, you might move from city-level data to country-level data.
- Drill-down: The reverse of roll-up, drill-down lets you navigate from higher-level data to more detailed data. You can move from quarterly data to monthly data, for example.
- Slice: Slicing involves selecting a specific portion of data from a single dimension. You might select data for a particular quarter.
- Dice: Dicing is more powerful as it allows you to select data using multiple dimensions. You could choose data for a specific location, quarter, and product category.
- Pivot: Pivot, also known as rotation, rearranges data axes to provide alternative presentations. For example, you could change from viewing data in a 2D slice to 2D planes.
- These operations enable users to analyze data from different angles and levels of detail, enhancing their data exploration capabilities.

Indexing OLAP Data

- Efficient data access is essential in OLAP environments to ensure fast query response times.
- Bitmap Indexing: This method uses bit vectors to represent data values. It's particularly useful for low-cardinality domains and can significantly reduce processing time and space requirements. Example 3.5 demonstrates its efficiency.
- Join Indexing: In scenarios where you need to join fact and dimension tables frequently, join indexing precomputes and stores identifier pairs of join results, making subsequent joins more efficient.
- These indexing techniques are vital for improving query performance in OLAP systems.

Column-Based Databases

- Traditional relational databases store data row by row, which is inefficient for selective queries.
- Column-based databases store data by column, allowing for substantial storage space and I/O savings.
- For instance, if you need to compute averages for specific attributes, a column-based database only reads the columns of interest, drastically reducing I/O.
- This approach is widely adopted in industry data warehousing and OLAP databases due to its efficiency in handling aggregate queries.
- Example 3.8 provides a practical illustration of the advantages of column-based storage.

Data Cubes

Table 3.1 2-D view of sales data according to *time* and *item*.

	location = "Vancouver"			
time (quarter)	item (type)			
	home entertainment	computer	phone	security
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is dollars_sold *(in thousands).*

Table 3.2 3-D view of sales data according to *time*, *item*, and *location*.

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
time	item				item				item				item			
	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is dollars_sold *(in thousands).*

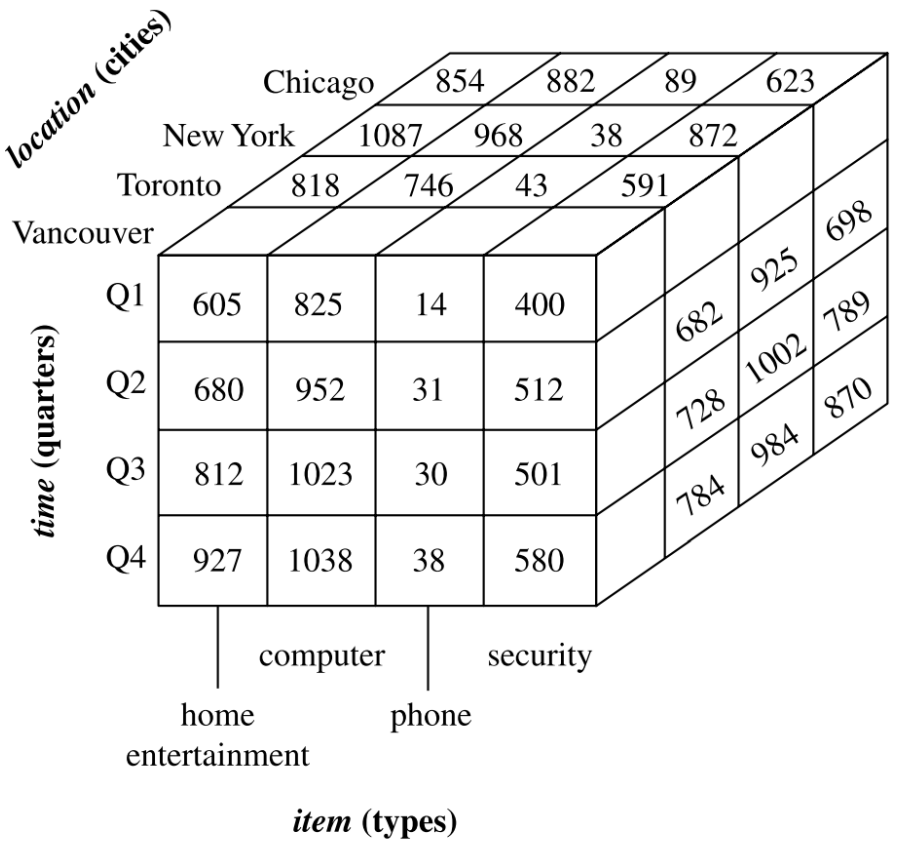


FIGURE 3.4

A 3-D data cube representation of the data in Table 3.2, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

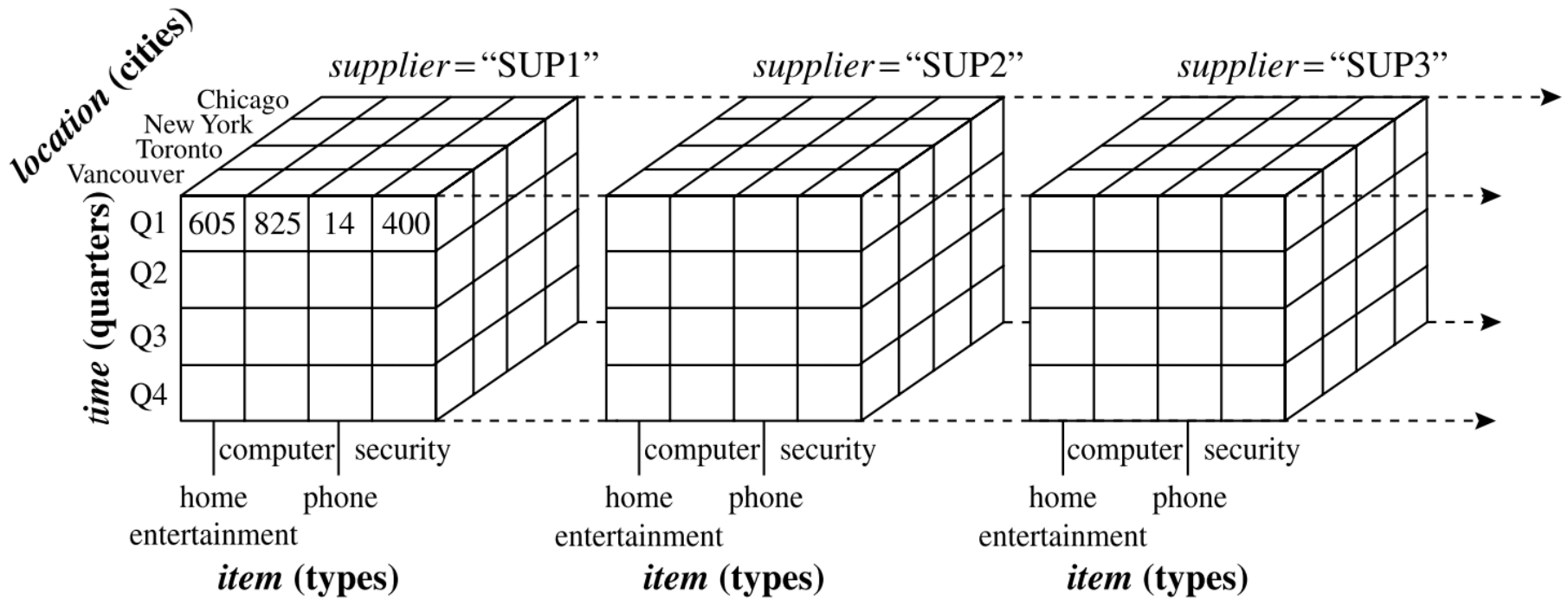


FIGURE 3.5

A 4-D data cube representation of sales data, according to *time*, *item*, *location*, and *supplier*. The measure displayed is *dollars_sold* (in thousands). For improved readability, only some of the cube values are shown.

Data Cube Computation

Data warehouses contain vast data volumes, and OLAP servers need to provide rapid responses to decision support queries.

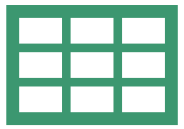
Data cubes are central to data warehouses, necessitating efficient data cube computation, access, and query processing.

This section provides an overview of data cube computation and introduces key concepts and strategies.

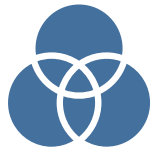
Terminology of Data Cube Computation

- Data cubes organize data into multidimensional structures.
- A cell in a cuboid represents a point in the data cube.
- Cells can be base or aggregate cells, depending on the level of aggregation.
- Ancestor–descendant relationships exist between cells based on dimensions.
- The number of cuboids in an n-dimensional data cube can be calculated considering hierarchies.

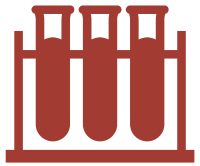
Data Cube Materialization: Ideas



Data cube materialization involves precomputing cuboids for analysis.



Three options for materialization: No Materialization, Full Materialization, and Partial Materialization (e.g., Iceberg Cubes).



Iceberg Cubes store only cells meeting specific criteria, reducing storage and speeding up analysis.

Efficient methods are necessary for both full and partial materialization.

OLAP Server Architectures

- OLAP server architectures: ROLAP, MOLAP, HOLAP, and Specialized SQL Servers.
- ROLAP uses relational tables, MOLAP uses multidimensional arrays, and HOLAP combines both.
- Specialized SQL Servers offer advanced query support for star and snowflake schemas.

General Strategies for Data Cube Computation

OPTIMIZATION TECHNIQUE 1: SORTING, HASHING, AND GROUPING FOR EFFICIENT DATA ACCESS.

OPTIMIZATION TECHNIQUE 2: SIMULTANEOUS AGGREGATION AND CACHING OF INTERMEDIATE RESULTS TO REDUCE I/O OPERATIONS.

OPTIMIZATION TECHNIQUE 3: AGGREGATION FROM THE SMALLEST CHILD WHEN MULTIPLE CHILD CUBOIDS EXIST.

OPTIMIZATION TECHNIQUE 4: PRUNING SEARCH SPACE USING THE DOWNWARD ANTIMONOTONICITY PROPERTY IN ICEBERG CUBE COMPUTATION.

Data Cube Computation Methods

- Data cube computation is essential for data warehouse implementation.
- Precomputation of data cubes can enhance performance and reduce response times.
- Efficient methods are crucial due to computational and storage challenges.
- This section explores optimization strategies for data cube computation.

Multiway Array Aggregation

- Multiway Array Aggregation (MultiWay): Computes a full data cube.
- Utilizes multidimensional arrays for efficient computation.
- Key Steps:
  - Partition the array into chunks to fit memory.
  - Aggregate by visiting cube cells with optimized order.
- Example: Explains MultiWay with a 3-D data array.
- Highlights memory optimization for different dimensional planes.

BUC Algorithm for Iceberg Cube Computation

- BUC (Bottom-Up Construction): Computes sparse and iceberg cubes.
- Works top-down in the cube lattice.
- Utilizes downward antimonotonicity property for pruning.
- Algorithm Overview:
  - Aggregate input and write the total.
  - Partition dimensions and test for minimum support.
  - Recursively compute iceberg cubes based on partition results.
- Importance of dimension order and data uniformity.

Precomputing Shell Fragments for High-Dimensional OLAP

MATERIALIZING DATA CUBES ENHANCES OLAP OPERATIONS.

CHALLENGES IN COMPUTING HIGH-DIMENSIONAL DATA CUBES INCLUDE STORAGE AND COMPUTATION TIME.

EXISTING METHODS ARE LIMITED TO LOW-DIMENSIONAL DATA.

INTRODUCTION TO THE CONCEPT OF SHELL FRAGMENTS FOR HIGH-DIMENSIONAL OLAP.

Shell Fragment Approach

- Most OLAP operations involve only a subset of dimensions relevant to a query.
- Shell fragment approach focuses on low-dimensional cuboids within a high-dimensional cube.
- Precompute cube shell fragments based on dimension partitioning.
- Assemble low-dimensional cubes online for fast OLAP using inverted index data structure.

Computing Shell Fragments

- Partition dimensions into disjoint fragments.
- Compute full local data cubes while retaining inverted indices for each fragment.
- Example of fragment partitioning for a 60-dimensional cube.
- Shell fragments drastically reduce the number of cuboids to compute compared to a full cube shell.

Conclusion

- Data warehouses serve management decision-making, distinct from operational databases.
- They adopt a three-tier architecture with warehouse database, OLAP server, and client tools.
- Metadata defines warehouse objects, including structure, history, and business terms.
- Data lakes store enterprise data in its natural format, complementing data warehouses.

Conclusion

- Multidimensional models (e.g., star, snowflake) organize data cubes with facts and dimensions.
- Cuboids within data cubes represent different levels of data summarization.
- Concept hierarchies organize attribute values into abstraction levels.
- OLAP operations (roll-up, drill-down, etc.) leverage data cube structures for efficiency.

Conclusion

- Index structures (e.g., bitmap, join indexes) enhance data access in data warehouses.
- Column-based databases optimize data storage by columns, reducing I/O costs.
- Materialization options for data cubes include full, partial (e.g., iceberg cubes, shell fragments), and quotient cubes.
- Efficient data cube computation methods include multi-way aggregation, BUC, and shell-fragment cubing.